ELI MKXI — Coverage Increase Analysis

Contents

Question: can the 48.1% testable coverage go higher, and if so, where? **Short answer:** yes — a focused pass on *testable-but-untested logic* can realistically reach **~54–56%**. Beyond that is a hard floor of code that cannot be honestly tested in CI (side-effecting OS actions, physical hardware, GPU, the un-buildable main window, and other-OS branches). This document breaks down, per area, **what brings the score down** and **what could bring it up**, with realistic estimates.

Baseline 2026-07-02: **48.1%** (33,681 / 70,086). **Update 2026-07-03 — the shortlist pass below was executed: now 49.2% (34,514 / 70,086), 7,481 tests passing.** The remaining path to the mid-50s (more logic tests + the planner/effector refactor) still stands.

---

1. The taxonomy of uncovered code

Every uncovered line falls into one of four buckets. Only two of them are honestly closeable:

Bucket	Closeable?	What it is
A. Testable logic, untested	[yes] yes	Pure decision/parse/format code no one wrote a test for yet
B. Partially testable	[!] some	Needs the live model / real sqlite / offscreen Qt — coverable but slower
C. Side-effecting / hardware / GPU / network	[no] no	Running it *does the thing* (opens apps, drives the mic, hits the net)
D. Other-OS / dead-defensive	[no] no*	`if WINDOWS/darwin …` branches; `except` fallbacks for impossible states

* Bucket D is coverable only by *mocking* the platform — which tests the branch selector, not real behaviour. Noted where it applies; not counted as an honest gain.

---

2. The biggest single opportunities (ranked, testable)

File	Now	Uncovered	Type	Realistic target	Gain
`runtime/deterministic_grounding_gate.py`	24%	1,504	A — layered ren- der_action + branches	60%	**+540**
`kernel/engine.py`	51%	3,179	B — needs varied live turns	58%	**+460**
`runtime/grounded_remediation.py`	21%	664	A — in- tent/state logic (+ some darwin)	55%	**+230**
`memory/memory.py`	60%	770	B — store/recall/KG with real sqlite	74%	**+230**
`cognition/gguf_inference.py`	55%	661	A/B — template detect + config	70%	**+220**
`api/server.py`	51%	570	A — POST/mutating/error endpoints	68%	**+200**
`execution/router_enhanced.py`	70%	803	A — more routing intents	85%	**+200**
`runtime/self_improvement.py`	40%	424	A — pro- posal/patch logic	65%	**+180**
`gui/labs_tab.py`	40%	1,848	B — invoke slots via QTest	52%	**+360**
`cognition/orchestrator.py`	51%	230	A — DAG scenarios	75%	**+110**
`planning/proactive_daemon.py`	15%	596	A/C — extract decision logic from loops	40%	**+150**
`tools/image_engine/…/engine.py`	0%	989	B — the *procedu- ral* (non-GPU) path	30%	**+300**
Smaller logic files (re- search_corpus, startup_hardware_optimizer, news_synthesis, de- vice_server, agent_bus…)	—	~1,400	A	—	**+700**

Sum of realistic gains ≈ +3,900 statements → ~54–56% overall.

---

3. Per-area breakdown — what's down, what's up

`execution` — 41% (12,309 stmts) · biggest drag

- **Down:** `executor_enhanced.py` (6,142 uncovered) — ~174 action handlers that *do things*: OPEN_APP, SHELL_EXEC, SCREENSHOT, VOLUME, media/keyboard/mouse, file writes. Running them in a test performs the action. This one file is ~26% of *all* uncovered code.
- **Up:** more **safe read-only live-lane turns** (already added date/gpu/list/read/ create); the *pure decision/validation* portions of handlers could be extracted into helpers and unit-tested. Router beside it is already **70%**.
- **Verdict:** mostly Bucket C. Realistic 41% → ~48%.

`runtime` — 51% (13,131 stmts) · biggest *opportunity*

- **Down:** `deterministic_grounding_gate.py` (24%) — the layered `render_action` wrappers + error branches; `grounded_remediation.py` (21%); `device_server/drivers` (MQTT hardware); `self_improvement.py` (40%).
- **Up:** the grounding gate and remediation are **pure logic** — the single richest vein left. Table-driven tests over action types, intents, and states would move both a lot. Device paths need a broker (partial).
- **Verdict:** Bucket A-heavy. Realistic 51% → ~62%.

`kernel` — 52% (7,059 stmts)

- **Down:** `engine.py` (3,179 uncovered) — pipeline branches that only fire on specific intents, modes, or escalation paths through the **live model**.
- **Up:** more varied **live-lane turns** (deeper modes, more intents, error inputs, escalation) — each real turn lights up hundreds of lines.
- **Verdict:** Bucket B. Realistic 52% → ~58%.

`cognition` — 65% (7,021 stmts)

- **Down:** `gguf_inference.py` (needs a model for generation paths), some agent/persona branches.
- **Up:** template-detection + config + no-think/think-strip logic are testable without a model; more agent/reranker unit tests. `agent_bus` already **79%**.
- **Verdict:** mixed. Realistic 65% → ~72%.

`gui` — 45% (5,439 stmts) · already recovered from 0.6%

- **Down:** the **main window** (excluded — hangs headless) and **event handlers** that only run on real clicks/signals (labs_tab 1,848 uncovered).
- **Up:** drive widget **slots/handlers directly** or via `QTest` signal simulation on the already-constructed widgets — this reaches the handler bodies without a human.
- **Verdict:** Bucket B. Realistic 45% → ~55%.

`tools` — 30% (4,508 stmts)

- **Down:** `image_engine` (989 + 795 uncovered) — GPU diffusion + visual core; news/weather fetchers (network, gated off).
- **Up:** the **procedural** image path (no GPU) is testable; more `news_synthesis` logic (formatting/ranking) is testable offline.
- **Verdict:** mixed. Realistic 30% → ~42%.

`perception` — 27% (4,138 stmts) · mostly floored

- **Down:** `audio_stt`, `vision`, `os_controller`, `wakeword` — mic/camera/GPU/live- desktop. Genuinely untestable headless.
- **Up:** little left — the pure parsers are already done (equations 100%, CSV 78%).
- **Verdict:** Bucket C. Realistic 27% → ~30% (near its honest ceiling).

`memory` — 54% (3,338 stmts)

- **Down:** `memory.py` (770 uncovered) — FAISS/embedding paths + some knowledge-graph ops.
- **Up:** more store/recall/dedup/KG tests against the real in-project sqlite (fast, isolated). FAISS paths need the vector backend.
- **Verdict:** Bucket A/B. Realistic 54% → ~68%.

`core` — 55% (3,492 stmts)

- **Down:** `model_download` (network), `startup_hardware_optimizer` (231), platform branches.
- **Up:** hardware-optimizer math + config/paths logic are pure and testable.
- **Verdict:** Bucket A. Realistic 55% → ~64%.

`planning` — 42% (2,132 stmts)

- **Down:** `proactive_daemon.py` (15%) — background timer loops tests don't drive.
- **Up:** extract the daemon's **decision logic** from the loop bodies and test it directly; scheduler/`infer_kind`/`parse_when` already covered.
- **Verdict:** Bucket A/C. Realistic 42% → ~55%.

`api` — 49% (1,204 stmts)

- **Down:** `server.py` (570 uncovered) — POST/mutating endpoints, error branches, the new model-switch path. (The embedded PWA JavaScript isn't counted as Python.)
- **Up:** more endpoint tests through the web lane — mutating routes with auth, 4xx paths, the settings/model endpoints.
- **Verdict:** Bucket A. Realistic 49% → ~66%.

`utils` — 25% (468 stmts)

- **Down:** `platform_compat.py` (350 uncovered) — `if WINDOWS/darwin …` branches that never run on Linux CI.
- **Up: only** by monkeypatching `platform.system()` to force each branch — that tests branch *selection*, not real cross-OS behaviour. Borderline (Bucket D).
- **Verdict:** honest ceiling is low on Linux; ~25% → ~40% only with platform mocking.

High already — marginal headroom

`onboarding` 85%, `world` 85%, `coding` 81%, `learning` 62%, `contracts` 44%, `plugins` 39%, `system` 37%, `integrations` 26% (mostly stub). A few hundred combined.

---

4. The hard floor — what genuinely can't be covered

Roughly **~11,000 statements** are honestly untestable in CI and cap the ceiling:

- **Side-effecting handlers** (`executor_enhanced` bodies) — a test would drive the machine.
- **Physical hardware** — mic (`audio_stt`, `wakeword`), camera/vision (`vision`, `visual_core`), speakers (`tts_router`), gaze, `os_controller`, screenshots.
- **GPU** — image diffusion, vision models.

- **Network** — news/weather fetchers, model downloads (gated off by netguard in tests).
- **The main window** — `eli_pro_audio_gui_MKI.py` (~7k), hangs on device/display init.
- **Other-OS branches** — Windows/macOS code paths in `platform_compat` and scattered `if darwin/win32` guards.
- **Defensive `except` fallbacks** for states that can't occur in a test.

Forcing any of these "green" means faking the OS/hardware/network — which tests the mock, not ELI. That is deliberately **not** done here.

---

5. Verdict & recommendation

	Coverage
Today (after the shortlist pass)	**49.2%**
After a focused Bucket-A/B pass (grounding gate, remediation, engine turns, memory, api, gui slots, orchestrator, image procedural path, misc logic)	**~54–56%**
Theoretical max without faking the world	**~58–60%**
Hard floor of untestable code	**~16%** of the tree

Recommendation: the highest-value next pass, in order, is: 1. `deterministic_grounding_gate` + `grounded_remediation` (pure logic, +~770) 2. More **live-lane turns** for `engine.py` (+~460) 3. `api/server.py` mutating/error endpoints (+~200) 4. `memory.py` store/recall/KG (+~230) 5. `gui` slot/handler invocation via QTest (+~360)

That's ~+2,000 statements for well under the effort of chasing the untestable tail, and lands the honest number in the **mid-50s** — an exceptional figure for a fully-embodied, local, multimodal system where a sixth of the code physically cannot run in CI.